



REDES NEURONALES Y APRENDIZAJE PROFUNDO

Nombres: MACÍAS MONDRAGÓN MARCOS RODLFO, REYES CRUZ ROBERTO MISAEAL

Especialidad: Inteligencia Artificial

Fecha de la práctica: 17 de abril de 2026

Nombre de la práctica: Implementación y Uso de una Red ART para Clasificación de Vinos

Objetivo

Implementar y entrenar una red ART (Adaptive Resonance Theory) en Python para clasificar diferentes tipos de vino, basándose en algunas de sus propiedades. Esta práctica permitirá entender cómo la red ART clasifica patrones y ajusta sus pesos en función de los parámetros clave.

1. Contexto del Conjunto de Datos:

- Se utilizará una versión simplificada del conjunto de datos de vino con solo tres características:
 - **Acidez** (nivel de ácido málico).
 - **Color** (tonalidad de color).
 - **Magnesio** (contenido de magnesio).
- Cada vino tendrá distintas características que lo harán clasificarse en categorías.

2. Carga y Preparación de los Datos

Los datos pueden ser descargados de `sklearn.datasets`.

```
from sklearn.datasets import load_wine
wine_data = load_wine()
```

3. Implementación de la Red ART

Definimos los parámetros básicos y los pesos iniciales para la red ART:

- **Umbral de resonancia:** $\theta=0.7$
- **Tasa de aprendizaje:** $\alpha=0.2$

4. Clasificación y Aprendizaje

Ahora implementamos la lógica de categorización y ajuste de pesos. Para cada patrón:

1. Calcula la activación de la categoría, si no existe creala
2. Calcula la **similitud de coseno** con cada neurona de salida.
3. Si la similitud con una neurona cumple el umbral de resonancia, clasifica el patrón en esa categoría y ajusta los pesos.

4. Si ninguna neurona cumple el umbral, agrega una nueva neurona con los valores del patrón como pesos iniciales.

5. Experimentos y Observaciones

1. **Clasificación:** Observa cómo se agrupan los patrones en diferentes categorías según el umbral de resonancia y la tasa de aprendizaje.
2. **Variación de Parámetros:**
 - Cambia el **umbral de resonancia** (θ) a otros valores para ver cómo afecta la creación de nuevas categorías.
 - Cambia la **tasa de aprendizaje** (α) para ver cómo afecta el ajuste de pesos.
3. **Visualización de Pesos y Categorías:** Representa las categorías y los pesos después del entrenamiento para observar cómo se agrupan los patrones.

Muestra la evidencia de estos experimentos.

6. Reflexión

1. ¿Cómo afecta el umbral de resonancia a la precisión y cantidad de categorías creadas?
2. ¿Qué efectos tiene la tasa de aprendizaje en la velocidad de ajuste y precisión de la red?
3. ¿Los resultados de la clasificación en cuanto a número de categorías y categoría asignada son congruentes con el tipo de vino del dataset?

7. Conclusiones

- La aplicación de la red ART ¿Cómo es útil en datos sin etiquetas previas?.

Parte 1 y 2: Se hace la importación de las librerías correspondientes y se seleccionan los parámetros deseados.

```
# =====  
# Parte 1 y 2: Contexto y Preparación de Datos  
# =====  
import numpy as np  
from sklearn.datasets import load_wine  
from sklearn.preprocessing import MinMaxScaler  
import matplotlib.pyplot as plt  
from mpl_toolkits.mplot3d import Axes3D  
  
wine_data = load_wine()  
# Seleccionar 3 características: Acidez, Color, Magnesio  
X = wine_data.data[:, [0, 9, 10]]  
# Normalizar entre 0 y 1 (requisito para ART1)  
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X)
```

✓ 1.9s

Figura 1. Código inicial.

Parte 3 y 4: Se implementa la clase ART, donde se describe el comportamiento de la Red, definiendo los inputs de *alpha* y *theta*. Inicialmente sin neuronas. Con las funciones de Similaridad de cose, el mach, train y la predicción.

```
# Parte 3: Implementación de la Red ART
#
class ART:
    def __init__(self, input_dim, theta=0.7, alpha=0.2):
        self.theta = theta # Umbral de resonancia
        self.alpha = alpha # Tasa de aprendizaje
        self.input_dim = input_dim
        self.weights = np.zeros((0, input_dim)) # Inicialmente sin neuronas

    def similarity(self, pattern, weight):
        # Similitud de coseno
        return np.dot(pattern, weight) / (np.linalg.norm(pattern) * np.linalg.norm(weight) + 1e-6)

    def match(self, pattern):
        if len(self.weights) == 0:
            return -1
        sims = [self.similarity(pattern, w) for w in self.weights]
        idx = np.argmax(sims)
        if sims[idx] >= self.theta:
            return idx
        return -1

    def train(self, patterns, epochs=1):
        for _ in range(epochs):
            for p in patterns:
                idx = self.match(p)
                if idx == -1:
                    self.weights = np.vstack([self.weights, p])
                else:
                    self.weights[idx] = self.alpha * p + (1 - self.alpha) * self.weights[idx]

    def predict(self, patterns):
        return [self.match(p) for p in patterns]
```

Figura 2. Código del ART.

Se inicia la Clasificación el el aprendizaje con los parámetros solicitados en la práctica(véase el código en la Figura 3). Al ejecutar, nos da como resultado la detección de dos clases; de Categoría 0 y -1, como el ejemplo que se adjunta en la Figura 4.

```
# Parte 4: Clasificación y Aprendizaje
#
art = ART(input_dim=X_scaled.shape[1], theta=0.7, alpha=0.2)
art.train(X_scaled, epochs=5)
predictions = art.predict(X_scaled)

print("Número de categorías detectadas:", len(set(predictions)))
for i, p in enumerate(predictions):
    print(f"Vino {i+1}: Categoría {p}")
```

Figura 3. Código de clasificación y Aprendizaje con los parámetros solicitados.

```
Vino 57: Categoría 0
Vino 58: Categoría 0
Vino 59: Categoría 0
Vino 60: Categoría -1
Vino 61: Categoría -1
Vino 62: Categoría 0
Vino 63: Categoría 0
Vino 64: Categoría -1
Vino 65: Categoría -1
Vino 66: Categoría 0
Vino 67: Categoría 0
Vino 68: Categoría 0
Vino 69: Categoría 0
Vino 70: Categoría -1
Vino 71: Categoría 0
Vino 72: Categoría 0
Vino 73: Categoría 0
Vino 74: Categoría -1
Vino 75: Categoría -1
Vino 76: Categoría -1
Vino 77: Categoría 0
Vino 78: Categoría -1
Vino 79: Categoría 0
Vino 80: Categoría -1
Vino 81: Categoría -1
Vino 82: Categoría 0
Vino 83: Categoría -1
Vino 84: Categoría 0
Vino 85: Categoría 0
Vino 86: Categoría -1
```

Figura 4. Extracto de los resultados obtenidos con los parámetros especificados.

Parte 5: Toman 3 variaciones de cada uno de los umbrales (vease Figura 5.) En la Figura 6, podemos ver las variaciones, y resaltamos que en el caso de θ a partir de 0.85 se detectan mas categorías y en el caso de α no se detectan mas categorías.

```
# Parte 5: Experimentos y Observaciones

# 1. Variación de umbral de resonancia  $\theta$ 
print("\n--- Variación de  $\theta$  ---")
thetas = [0.6, 0.7, 0.85]
for t in thetas:
    art_temp = ART(input_dim=X_scaled.shape[1], theta=t, alpha=0.2)
    art_temp.train(X_scaled, epochs=5)
    pred_temp = art_temp.predict(X_scaled)
    print(f" $\theta = {t}$  → Categorías detectadas: {len(set(pred_temp))}")

# 2. Variación de tasa de aprendizaje  $\alpha$ 
print("\n--- Variación de  $\alpha$  ---")
alphas = [0.1, 0.5, 0.75]
for a in alphas:
    art_temp = ART(input_dim=X_scaled.shape[1], theta=0.7, alpha=a)
    art_temp.train(X_scaled, epochs=5)
    pred_temp = art_temp.predict(X_scaled)
    print(f" $\alpha = {a}$  → Categorías detectadas: {len(set(pred_temp))}")

# 3. Visualización 3D de clusters y pesos
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
colors = ['r', 'g', 'b', 'c', 'm', 'y', 'k']

for i, p in enumerate(predictions):
    ax.scatter(X_scaled[i, 0], X_scaled[i, 1], X_scaled[i, 2],
              color=colors[p % len(colors)], label=f"Categoría {p}" if i==0 else "")

# Mostrar pesos de cada categoría
for idx, w in enumerate(art.weights):
    ax.scatter(w[0], w[1], w[2], color='k', marker='x', s=100, label=f"Peso {idx}" if idx==0 else "")

ax.set_xlabel("Acidez")
ax.set_ylabel("Color")
ax.set_zlabel("Magnesio")
plt.title("Clusters y Pesos de la Red ART")
plt.show()
```

Figura 5. Código de los experimentos realizados.

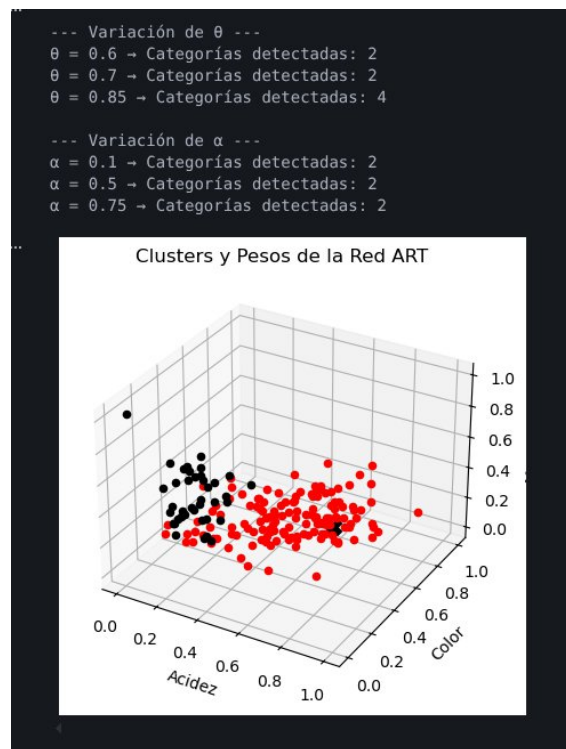


Figura 6. Los resultados de los experimentos realizados.

La gráfica muestra dos clusters bien definidos en el espacio de acidez, color y magnesio. Un grupo es compacto (negro), indicando baja variabilidad, mientras que el otro (rojo) está más disperso, lo que refleja mayor diversidad en sus características.

La separación entre ambos grupos indica que las variables utilizadas permiten distinguir patrones de manera efectiva. Esto coincide con los resultados donde, para valores de $\theta = 0.6$ y 0.7 , la red identifica 2 categorías. Cuando θ aumenta a 0.85, se generan más categorías, lo que sugiere que la red comienza a subdividir los grupos existentes debido a un criterio de similitud más estricto.

Parte 6:

1. ¿Cómo afecta el umbral de resonancia a la precisión y cantidad de categorías creadas? Un umbral alto hace que la red sea más estricta, por lo que se crean más categorías (mayor precisión pero más fragmentación). Un umbral bajo agrupa más datos en menos categorías, pero puede mezclar patrones diferentes.
2. ¿Qué efectos tiene la tasa de aprendizaje en la velocidad de ajuste y precisión de la red? Una tasa alta hace que los pesos cambien rápidamente, lo que puede volver el aprendizaje inestable. Una tasa baja hace el aprendizaje más lento, pero más preciso y estable.

3. ¿Los resultados de la clasificación en cuanto a número de categorías y categoría asignada son congruentes con el tipo de vino del dataset? Los resultados son parcialmente congruentes con el dataset. La gráfica muestra dos clusters diferenciados, lo que coincide con que la red detecta 2 categorías. Sin embargo, en el output casi todos los vinos se asignan a la categoría 0, mientras que varios aparecen como -1 (no clasificados o sin resonancia).

Esto significa que la red sí distingue una separación general, pero no está distribuyendo correctamente los datos entre las categorías, ya que una domina casi por completo.

Deberíamos de haber encontrado 3 categorías mas una donde no reconozca nada (osease 4).

El hecho de que muchos patrones queden en una sola categoría o sin clasificar sugiere que el modelo no está capturando completamente la estructura real de las clases.

En conclusión, hay coherencia visual en los clusters, pero la asignación de categorías no refleja adecuadamente la diversidad del dataset.

Parte 7:

La aplicación de la red ART ¿Cómo es útil en datos sin etiquetas previas?.

La red ART es útil para clasificar datos sin etiquetas porque puede crear categorías automáticamente basadas en similitud.

Esto la hace adecuada para problemas donde no se conoce previamente la estructura de los datos.